Zewail City of Science and Technology

School of Computational Sciences and Artificial Intelligence (CSAI)

Graduation Project Report

Chest X-ray Pneumonia Detection System

An AI-Assisted Web Platform for Pneumonia Screening and Localization from Chest Radiographs

Submitted by

Student Name	Student ID	Program
Moamen Elsayed Elsharkawy	202202015	CSAI
Habiba Ayman Amin	202202088	CSAI
Ahmed Gamal Abdelfattah	202201638	CSAI
Sara Mostafa Ali	202201305	CSAI

Supervisor: Prof. Dr. Khaled Mostafa

Team Number: 19 | **Academic Program:** CSAI

Submitted in Partial Fulfillment of the Requirements for the Degree of

Bachelor of Science in Computational Sciences and Artificial Intelligence

Date: June 2026

Abstract

Pneumonia remains one of the leading causes of illness and death worldwide, and the chest X-ray is the first-line tool used to detect it. In many clinics, however, the number of radiographs far exceeds the number of radiologists available to read them, which delays diagnosis and increases the chance of human error and missed cases. This project presents the Chest X-ray Pneumonia Detection System, a complete, doctor-facing web application that uses deep learning to support pneumonia screening from chest radiographs. The system lets a physician register, create patient records, upload an X-ray image, and receive an automated reading that includes whether pneumonia is likely present, where the suspicious region is located as a bounding box, a numeric confidence score, and a visual heatmap that explains the model decision. Every result is stored per patient so it can be reviewed later.

The platform is a modular three-tier system: an Angular single-page frontend, an asynchronous FastAPI backend, and a MongoDB database, with a PyTorch inference layer that serves five trained models through one registry: two detectors that localize pneumonia (Faster R-CNN and YOLOv8) and three classifiers (ResNet50, DenseNet121, and EfficientNet-B0). All models were trained and evaluated on the RSNA Pneumonia Detection Challenge dataset using a leak-free, patient-wise split. On the held-out test set the strongest classifier (EfficientNet-B0) reached an AUC of 0.886, and the deployed detector (Faster R-CNN) reached a recall of 0.812, the property that matters most clinically because it minimizes missed pneumonia. Authentication uses JSON Web Tokens with hashed passwords, and every prediction is paired with explainability so the clinician keeps control of the final decision. The result is a reproducible, end-to-end system that embeds modern AI into a practical clinical workflow rather than leaving it as an isolated model.

الملخص (Arabic Abstract)

يُعد الالتهاب الرئوي من أكثر الأمراض انتشارًا وخطورة على مستوى العالم، وتُعتبر أشعة الصدر السينية الأداة الأولى لاكتشافه. إلا أن أعداد صور الأشعة في كثير من المستشفيات تفوق بكثير عدد أطباء الأشعة المتاحين لقراءتها، مما يؤخر التشخيص ويزيد من احتمال الخطأ البشري وإغفال بعض الحالات. يقدم هذا المشروع نظام الكشف عن الالتهاب الرئوي من أشعة الصدر، وهو تطبيق ويب متكامل موجَّه للأطباء يستخدم التعلم العميق لمساعدتهم في فحص الالتهاب الرئوي من صور الأشعة. يتيح النظام للطبيب تسجيل الدخول وإنشاء ملفات المرضى ورفع صورة الأشعة، ثم يحصل على قراءة آلية تتضمن احتمالية وجود الالتهاب الرئوي، وموقعه على الصورة في صورة مربع تحديد، ودرجة ثقة رقمية، وخريطة حرارية توضح أساس قرار النموذج.

بُني النظام على ثلاث طبقات: واجهة أمامية باستخدام Angular، وخادم خلفي غير متزامن باستخدام FastAPI، وقاعدة بيانات MongoDB، إضافة إلى طبقة استدلال باستخدام PyTorch تقدم خمسة نماذج مدربة عبر سجل نماذج موحد. اثنان منها نموذجا كشف يحددان موقع الالتهاب (Faster R-CNN وYOLOv8)، وثلاثة نماذج تصنيف (ResNet50 وDenseNet121 وEfficientNet-B0). دُرّبت جميع النماذج وقُيّمت على بيانات تحدي RSNA باستخدام تقسيم خالٍ من تسرب البيانات على مستوى المريض. على مجموعة الاختبار، حقق أفضل نموذج تصنيف (EfficientNet-B0) مساحة تحت المنحنى بلغت 0.886، وحقق النموذج المنشور للكشف (Faster R-CNN) معدل استرجاع بلغ 0.812، وهو الأهم سريريًا لأنه يقلل من الحالات المُغفَلة. يعتمد التحقق من الهوية على رموز JWT مع تشفير كلمات المرور، وكل تنبؤ مصحوب بتفسير بصري يُبقي القرار النهائي بيد الطبيب.

Table of Contents

This table of contents, list of figures, and list of tables are generated automatically. In Microsoft Word, select all (Ctrl+A) and press F9, or right-click each and choose Update Field, to populate page numbers.

List of Figures

List of Tables

Chapter 1 - Introduction

1.1 Background

Healthcare systems everywhere are under growing pressure to read more medical images with fewer specialists. Chest radiography, the chest X-ray, is the most common imaging examination in the world because it is fast, inexpensive, and widely available. It is also the standard first step when a clinician suspects pneumonia, an infection that inflames the air sacs of the lungs and appears on an X-ray as a hazy region of increased opacity. The challenge is that reading these images correctly requires expert training, and that expertise is scarce. A single emergency department can generate hundreds of radiographs in a day, while only one or two radiologists may be on duty to interpret them.

Over the last decade, deep learning, and in particular convolutional neural networks, has reached a level of performance on medical images that makes computer-assisted reading genuinely useful. Rather than replacing the doctor, these models act as a fast second reader: they flag images that look abnormal, point to the region that drives their decision, and let the clinician focus attention where it matters. This project sits in exactly that space.

1.2 Problem Statement

Manual interpretation of chest X-rays for pneumonia is slow, depends on scarce expertise, and is vulnerable to fatigue and inter-reader variability. There is a need for a practical, accessible system that can screen chest radiographs automatically, highlight suspicious regions, quantify its confidence, and keep a reviewable record per patient, all while keeping the physician firmly in control of the final diagnosis.

1.3 Motivation

The motivation is both clinical and engineering. Clinically, an automated screening aid can shorten the time to diagnosis and reduce missed cases, which is the most dangerous type of error in pneumonia. From an engineering standpoint, most academic AI work stops at a model in a notebook. We wanted to show the full path: take research-grade models and deliver them inside a secure, usable, end-to-end product that a doctor could actually operate.

1.4 Proposed Solution

We built the Chest X-ray Pneumonia Detection System, a web application in which a physician signs in, manages patient records, uploads a chest X-ray, and receives an automated reading. The backend runs a deep-learning model that returns a pneumonia decision, a bounding box for localization, a confidence score, and an explainability heatmap. Results are persisted per patient. The system supports multiple models behind one registry, with Faster R-CNN selected as the deployed default because it gives the best localization recall.

1.5 Objectives

- Train and rigorously evaluate deep-learning models for pneumonia detection and localization on a public, professionally labeled dataset.
- Deliver a secure, multi-user web application with authentication, patient management, and persistent analysis history.
- Provide visual explainability with every prediction so the result is interpretable and trustworthy.
- Build the system so it is reproducible, documented, and maintainable, not a one-off script.

1.6 Scope of the Project

The system targets binary pneumonia screening (pneumonia versus normal) on adult chest radiographs, delivered as a decision-support tool for clinicians. It is not a certified medical device and does not make an autonomous diagnosis; the physician reviews and confirms every result. Image acquisition, hospital information system integration, and regulatory certification are outside the current scope and are discussed as future work.

1.7 Challenges Addressed

- A data leak in early preprocessing that produced unrealistically high scores, which we found and fixed with patient-wise splitting and uniform image processing.
- Serving several large models efficiently from one backend without reloading them on every request.
- Making detection results explainable, including for detectors where standard saliency methods do not apply directly.

- Integrating an asynchronous Python AI backend with a modern Angular frontend and a document database in a clean, secure way.

1.8 Contributions of the Work

- A reproducible, leak-free training and evaluation pipeline that benchmarks five architectures on the RSNA dataset.
- A production-style inference service with a model registry, single-load warmup, non-maximum suppression, and six explainability methods.
- A complete, secure full-stack application that turns those models into a usable clinical workflow.
- An honest evaluation that calibrates the results against the published literature rather than overstating them.

1.9 Report Organization

Chapter 2 presents the market and business case. Chapter 3 reviews related work and background. Chapter 4 describes the system design. Chapter 5 details the implementation. Chapter 6 covers testing and evaluation, and Chapter 7 presents and discusses the results. Chapter 8 addresses ethics, compliance, and standards. Chapter 9 concludes and outlines future work. References and two appendices, a user guide and supporting material, follow.

Chapter 2 - Market Visibility and Business Case

2.1 Market Relevance

Pneumonia is a high-volume, high-impact clinical problem. It is among the leading causes of hospitalization and death globally, and chest radiography is the workhorse imaging test used to triage it. The global medical-imaging AI market has grown rapidly precisely because imaging volume is rising faster than the supply of radiologists. Tools that help clinicians read more images, more consistently, and faster therefore address a real and growing demand.

2.2 Target Users and Stakeholders

- Primary users: general physicians, emergency and internal-medicine doctors, and radiologists who need a fast second read.
- Healthcare facilities: clinics and hospitals, especially in under-served regions where specialist radiologists are scarce.
- Patients: the ultimate beneficiaries through faster, more consistent screening.
- Health administrators: who care about throughput, turnaround time, and auditability.

2.3 Existing Market Gaps

Commercial chest-imaging AI exists, but it is often expensive, closed, and tied to specific hospital infrastructure. Many academic prototypes, on the other hand, never leave the notebook: they report a metric but provide no usable interface, no patient record, no security, and no explanation. The gap we target is a practical, transparent, end-to-end screening tool that is interpretable by design and simple to operate.

2.4 Competitive Analysis

Solution	Type	Localization	Explainability	Accessibility
Lunit INSIGHT CXR	Commercial	Yes	Heatmaps	Enterprise, paid
Aidoc	Commercial	Yes	Limited	Enterprise, paid
qure.ai qXR	Commercial	Yes	Heatmaps	Enterprise, paid
Typical academic model	Research	Sometimes	Rare	Notebook only
This project	Academic, full-stack	Yes (boxes)	6 methods	Open, self-hostable

Table 1. Comparison of our system with representative existing solutions.

2.5 Potential Impact, Innovation, and Sustainability

The system can reduce time-to-screening and act as a safety net that lowers missed-case rates, which has direct clinical value. Its innovation is not a single new algorithm but the combination of rigorous, leak-free evaluation, multi-model flexibility through a registry, and explainability on every prediction, all delivered as a working product. Because the models are self-hosted and run on commodity hardware (inference works on CPU), the running cost is low and the design is sustainable for resource-constrained settings.

2.6 Feasibility, Scalability, and Commercialization

Technically the project is already feasible: it is implemented and runs end to end. The architecture is stateless at the API layer, so it scales horizontally behind a load balancer, and the model can be moved to its own service for heavier loads. Commercially, the natural path is a hosted SaaS offering for clinics, or an on-premise deployment for hospitals with data-residency requirements, with the explainability and audit trail as differentiators.

Chapter 3 - Literature Review and Needed Background

3.1 Deep Learning for Chest X-ray Analysis

Convolutional neural networks (CNNs) learn hierarchical visual features directly from pixels and have become the standard approach for medical image classification. A landmark example is CheXNet, a 121-layer DenseNet trained on the ChestX-ray14 dataset, which reported radiologist-level performance on pneumonia detection and popularized transfer learning for chest radiographs. Subsequent work confirmed that ImageNet-pretrained backbones such as ResNet, DenseNet, and EfficientNet provide strong starting points that fine-tune well on smaller medical datasets.

3.2 Object Detection for Localization

Classification alone tells the clinician whether pneumonia is likely, but not where. Object detection adds localization. Two families dominate: two-stage detectors such as Faster R-CNN, which first proposes candidate regions and then classifies and refines them, and one-stage detectors such as the YOLO family, which predict boxes directly in a single pass and are faster but often less sensitive on small or diffuse findings. Both were used in solutions to the RSNA Pneumonia Detection Challenge, the dataset we adopt.

3.3 The RSNA Pneumonia Detection Challenge

The RSNA Pneumonia Detection Challenge dataset, curated by the Radiological Society of North America, contains roughly 26,684 frontal chest radiographs with expert bounding-box annotations of lung opacities consistent with pneumonia. It is widely used as a benchmark because it is large, professionally labeled, and provides both image-level labels and box annotations, which makes it suitable for classification and detection alike.

3.4 Explainable AI for Medical Imaging

Trust requires explanation. Gradient-based methods such as Grad-CAM, Integrated Gradients, and GradientSHAP attribute a prediction back to image regions; gradient-free methods such as Score-CAM, Eigen-CAM, and occlusion sensitivity perturb or analyze activations instead. For a clinical tool, these visual explanations are essential: they let the physician verify that the model is looking at the lungs and not at an irrelevant artifact.

3.5 Comparative Analysis and Positioning

Most prior work optimizes a single metric on a single model, and many report inflated numbers because of data leakage between train and test at the patient level. Our work differs in three ways: we benchmark five architectures under one consistent, leak-free protocol; we deploy the result inside a secure, usable application rather than a notebook; and we attach explainability to every prediction. We also report honest numbers and calibrate them against the literature, where RSNA detection mAP typically sits between 0.32 and 0.39.

Chapter 4 - System Design

4.1 Functional Requirements

ID	Requirement
FR-1	A doctor can register, log in, and log out securely.
FR-2	A doctor can create, view, update, and delete patient records.
FR-3	A doctor can upload a chest X-ray image for analysis.
FR-4	The system runs inference and returns a pneumonia decision, bounding box, confidence, and heatmap.
FR-5	Each analysis is saved and linked to its patient and owning doctor.
FR-6	A doctor can browse the history of past analyses.
FR-7	A doctor can only access their own patients and analyses.

Table 2. Core functional requirements.

4.2 Development Methodology and Data-Flow Design

The project followed an iterative, phase-based methodology that separates the research track (training and evaluating the models) from the engineering track (building the application) and then integrates the two through a shared model registry. The research track is organized as an eight-phase AI pipeline, data preparation, baseline training, hyperparameter optimization, retraining, explainability, evaluation, and a single-image demo, run once per model and illustrated in Figure 7. The engineering track followed an incremental full-stack lifecycle: requirements, an API contract, backend services, the frontend, and integration testing, with the model layer plugged in last.

Data collection and preprocessing convert the raw RSNA DICOM studies into uniform 8-bit PNG images, applying identical contrast handling (CLAHE) to every image regardless of its label to avoid leakage, then split them patient-wise into train, validation, and test sets (detailed in Chapter 6). At inference time the runtime data flow is linear: the doctor uploads an image, the backend validates it, the warmed model produces a probability or bounding boxes, non-maximum suppression removes overlapping boxes, an explainability heatmap is rendered, and the structured result is persisted and returned. This end-to-end flow is shown in Figure 2.

4.3 Use Cases and User Flow

The main use case is a single, linear clinical flow: the doctor authenticates, selects or creates a patient, uploads an X-ray, waits on a processing screen while inference runs, and then reviews

a result screen showing the annotated image, the confidence, and a recommendation. The result is stored and appears in the patient history. Figure 2 shows this data flow.

4.4 Non-Functional Requirements

Attribute	Target / Approach
Security	JWT authentication, bcrypt password hashing, per-user authorization, input validation.
Performance	Models loaded once at startup (warmup); async I/O for non-blocking requests.
Scalability	Stateless API behind a load balancer; database holds all state.
Reliability	Health endpoint; best-effort explainability that never crashes a prediction.
Maintainability	Modular routers, a model registry, and a reproducible AI pipeline.
Usability	Clean Angular UI with a guided, linear workflow.

Table 3. Non-functional requirements.

4.5 Architecture Design

The system follows a modular three-tier architecture. An Angular single-page application (with server-side rendering) is the presentation layer; an asynchronous FastAPI service is the application layer; and MongoDB is the data layer. A dedicated inference service inside the backend owns all model logic and loads weights from a model registry. Figure 1 shows the high-level architecture.

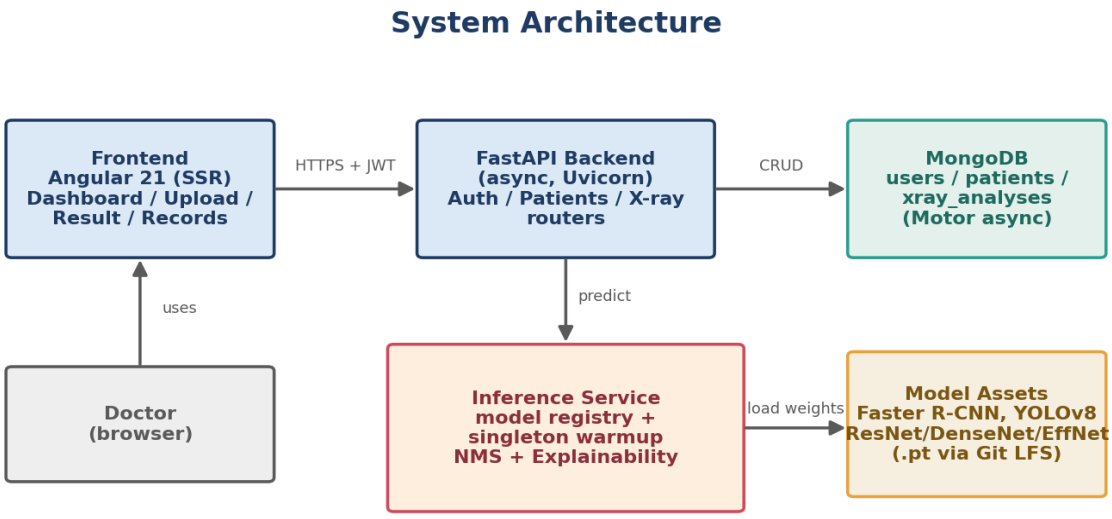

Figure 1. High-level system architecture (three tiers plus the inference service and model registry).

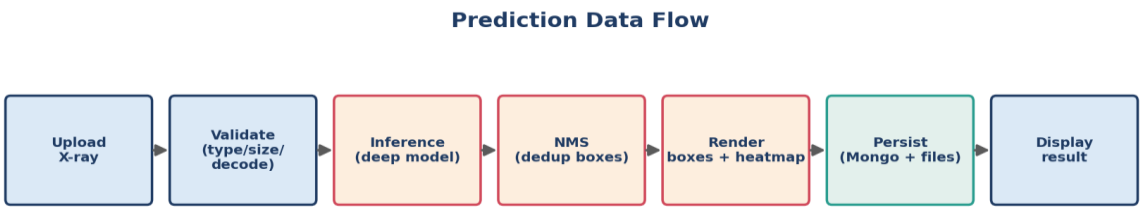

Figure 2. End-to-end prediction data flow, from upload to stored, displayed result.

4.6 Database Design

The database uses three MongoDB collections: users, patients, and xray_analyses. A user owns many patients, and each patient has many analyses. Unique indexes enforce email and identifier uniqueness, and a compound index on owner and creation time supports fast history queries. Figure 3 shows the entity relationships.

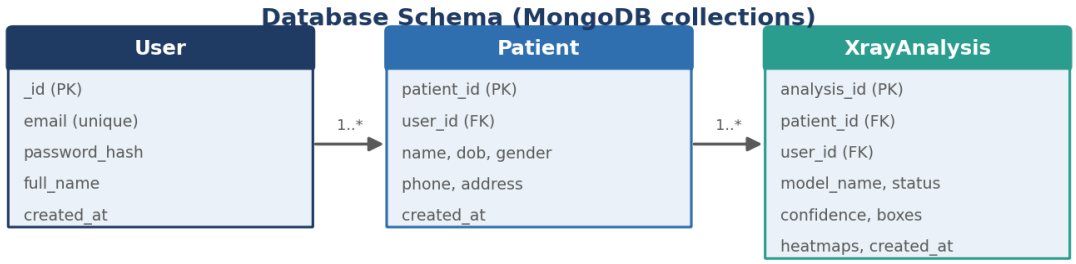

Figure 3. Entity relationship diagram for the three MongoDB collections.

4.7 API Design

Endpoint	Method	Purpose
/api/auth/signup, /login, /me	POST/GET	Authentication and current user.
/api/patients	CRUD	Patient record management.
/api/xray/upload	POST	Upload, run inference, persist result.
/api/xray, /api/xray/{id}	GET/DELETE	List and manage analyses.
/api/xray/status, /metadata	GET	Model status and metadata.
/health, /docs	GET	Health check and OpenAPI (Swagger) documentation.

Table 4. Main REST API endpoints. Full OpenAPI docs are auto-generated at /docs.

4.8 Deployment Architecture and Technology Stack

Layer	Technology
Frontend	Angular 21 (TypeScript), server-side rendering via Express
Backend	FastAPI (Python), Uvicorn ASGI server, async I/O
Database	MongoDB with the Motor async driver
AI / ML	PyTorch, torchvision, Ultralytics; FAISS-free, self-hosted weights
Auth & Security	JWT (python-jose), bcrypt (passlib), CORS
Model storage	Git LFS for the .pt checkpoints

Table 5. Technology stack by layer.

4.9 UI / UX Design

The interface is intentionally simple and linear so a busy clinician can move from upload to result with no training. After signing in, the doctor lands on a dashboard, opens the upload page (drag-and-drop with a live preview), waits on a short processing screen, and reviews a result screen showing the annotated image, the confidence score, and the heatmap. The patient-records and history views collect past analyses. Navigation is guarded so that protected pages require authentication and signed-in users are redirected away from the public pages. A sample annotated model output is shown in Figure 4, and the main screens of the running application are shown in Figures 5 and 6.

Figure 4. Sample detector output: the predicted pneumonia region drawn on a chest radiograph.

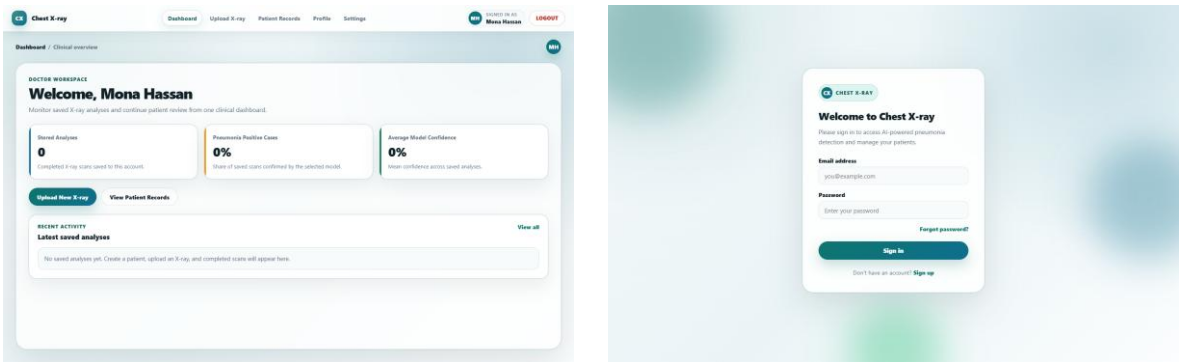

Figure 5. The doctor dashboard (left) and the secure login screen (right).

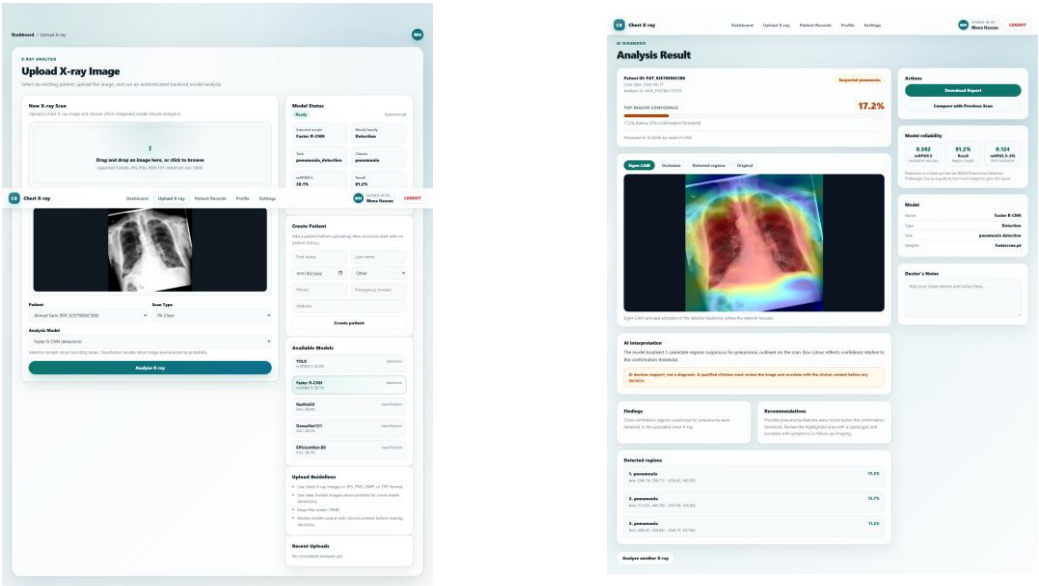

Figure 6. The X-ray upload page with model selector and live preview (left) and the result screen with the annotated image, confidence score, and heatmap (right).

Chapter 5 - Implementation Details

This chapter describes the main modules. For each, we give its purpose, the technologies used, its internal workflow, and the key engineering decisions.

5.1 Authentication Module

Purpose: secure, multi-user access. Technologies: FastAPI dependencies, python-jose for JSON Web Tokens, passlib with bcrypt for password hashing. Workflow: on signup the password is hashed with a per-password salt and never stored in plaintext; on login the server issues a signed JWT with an expiry; protected endpoints require a valid bearer token, which is decoded and verified on each request. A production guard refuses to start the server if it is run in production mode with the default secret key, preventing a common deployment mistake.

```
def create_access_token(data: dict, expires_delta=None):
    to_encode = data.copy()
    expire = datetime.now(UTC) + (expires_delta or
             timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str) -> dict:
    try:
        return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    except JWTError:
        raise HTTPException(status_code=401,
                            detail="Could not validate credentials")

# Production guard: refuse to boot with the default secret in production
if APP_ENV in {"prod", "production"} and (
        SECRET_KEY == DEFAULT_DEVELOPMENT_SECRET or len(SECRET_KEY) < 32):
    raise RuntimeError("SECRET_KEY must be strong and unique in production.")
```

Listing 1. JWT issuance and verification, with the production secret-key guard (Backend/app/utils/security.py).

5.2 AI / ML Inference Module

Purpose: turn an uploaded image into a clinical reading. The module is a single inference service that holds a model registry mapping a model key to its weights, task, and thresholds. At startup the default model is warmed up, that is loaded once into memory, so requests do not pay the load cost. A request validates the image, runs the model, applies non-maximum suppression (NMS) to remove overlapping duplicate boxes, renders the annotated image, generates an explainability heatmap, and returns a structured result. Faster R-CNN is the deployed default detector; the registry also exposes YOLOv8 and the three classifiers.

Key decisions: a single warmed model instance keeps latency low and memory predictable; NMS with an intersection-over-union threshold of 0.30 prevents duplicate boxes; and explainability is best-effort, meaning that if a heatmap method fails it is skipped rather than failing the whole prediction. Figure 7 shows the AI pipeline that produced these models.

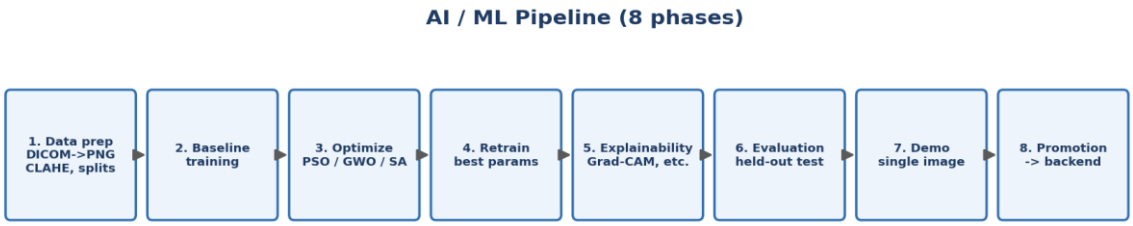

Figure 7. The eight-phase AI pipeline used to prepare, train, optimize, and deploy the models.

```
# Model registry: one validated entry per served model (key -> spec).
MODEL_CATALOG = {
    "fasterrcnn":      ModelCatalogSpec(family=DETECTION,      weights_file="fasterrcnn.pt",
default_conf=0.10),
    "yolo":            ModelCatalogSpec(family=DETECTION,      weights_file="yolo_best.pt",
default_conf=0.10),
    "efficientnet_b0": ModelCatalogSpec(family=CLASSIFICATION,
weights_file="efficientnet_b0.pt", input_size=224),
    "resnet50":        ModelCatalogSpec(family=CLASSIFICATION, weights_file="resnet50.pt",
input_size=224),
    "densenet121":     ModelCatalogSpec(family=CLASSIFICATION, weights_file="densenet121.pt",
input_size=224),
}

# Non-maximum suppression drops overlapping duplicate boxes, then sort by score.
keep = nms(boxes, scores, nms_iou_threshold).tolist()
detections = sorted((detections[i] for i in keep),
                    key=lambda d: d["confidence"], reverse=True)
```

Listing 2. The model registry and the non-maximum-suppression step in the inference service (Backend/app/utils/xray_inference.py).

The HTTP contract is a single multipart upload that returns the full structured reading. Listing 3 shows a representative request and response.

```
POST /api/xray/upload          (multipart: file, patient_id, model_key)
Authorization: Bearer <jwt>

200 OK
{
  "analysis_id": "ANA_3F9C12A0B7D4",
  "model": "fasterrcnn",
  "prediction": "pneumonia",
  "confidence": 0.81,
  "boxes": [{"x1": 286, "y1": 196, "x2": 470, "y2": 421, "score": 0.81}],
  "heatmap_url": "/uploads/ANA_3F9C12A0B7D4_gradcam.png",
  "created_at": "2026-06-16T12:04:33Z"
```

```
}
```

Listing 3. Representative upload request and the JSON reading returned by the API.

5.3 Backend Services

Purpose: orchestrate authentication, patient management, inference, and persistence. Technology: FastAPI with fully asynchronous request handling and a lifespan context that connects to MongoDB and warms the model on startup and cleans up on shutdown. Routers are split by resource (auth, patients, x-ray) for clarity, and Pydantic models validate every request and response.

5.4 Frontend Application

Purpose: the doctor-facing interface. Technology: Angular 21 with server-side rendering for fast first paint and better SEO on public pages. State is held in a central analysis-state service, routing is protected by authentication guards, and a single API service centralizes all backend calls so the rest of the app never talks to HTTP directly. The build is a standard Angular production build that prerenders the public routes.

```
// Route guard: only authenticated users reach protected pages.
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  if (authService.isAuthenticated()) return true;
  return router.createUrlTree(['/login']);
};
```

Listing 4. The Angular route guard that protects authenticated pages (Frontend/src/app/auth.guard.ts).

5.5 Database Layer

Purpose: durable storage of users, patients, and analyses. Technology: MongoDB through the asynchronous Motor driver. We chose a document database because one analysis is naturally a single nested document (boxes, scores, heatmap paths, timestamps) that is always read as a whole. Indexes are created at connection time to guarantee uniqueness and to make per-user history queries fast.

5.6 Hyperparameter Optimization

To tune the models we implemented three nature-inspired search algorithms: Particle Swarm Optimization (PSO), Grey Wolf Optimizer (GWO), and Simulated Annealing (SA). Each candidate configuration was scored with a fast proxy (few epochs on a data fraction) so the

search fit within a single GPU session, and the best configuration across the three was kept. As discussed in Chapter 7, this search did not beat a carefully tuned baseline, which is itself a meaningful engineering finding.

5.7 Security and Deployment Considerations

Secrets are read from environment variables and kept out of version control; large model weights are tracked with Git LFS; CORS is restricted to known origins; and uploads are validated for type, size, and decodability before they ever reach a model. The application is designed to be containerized and run behind a reverse proxy, which is described as the immediate next step in Chapter 9.

Chapter 6 - Testing and Evaluation

6.1 Testing Types

- Unit and integration tests for the backend (pytest), covering authentication and the full upload-to-history flow with a mocked database.
- Access-control tests proving that one doctor cannot read another doctor's patient.
- A preflight checker that validates that all required model assets and configuration are present before deployment.
- An end-to-end inference smoke test that exercises the live prediction path against a running backend.
- System and usability testing of the full clinical flow through the user interface.

6.2 Evaluation Metrics

For classification we report accuracy, precision, recall (sensitivity), specificity, F1, and the area under the ROC curve (AUC), together with the confusion matrix. For detection we report mean average precision at an IoU of 0.5 (mAP@0.5), mAP averaged over IoU thresholds from 0.5 to 0.95, recall, and precision. Recall is emphasized throughout because in pneumonia a false negative, a missed case, is the most costly error.

6.3 Experimental Setup

Models were trained on Kaggle GPU sessions (NVIDIA P100/T4) using PyTorch with mixed-precision training, early stopping on the validation metric, and standard data augmentation. The RSNA dataset was converted from DICOM to PNG with uniform contrast handling and split patient-wise into train, validation, and test sets with a fixed seed; the test set holds 20 percent of patients (4,135 normal and 1,202 pneumonia images) and is never seen during training or model selection.

Chapter 7 - Results and Discussion

7.1 Classification Results

Model	AUC	Accuracy	Recall	Specificity	F1	Params
ResNet50	0.884	0.810	0.761	0.824	0.644	23.5M
DenseNet121	0.883	0.802	0.785	0.807	0.641	7.0M
EfficientNet-B0	0.886	0.815	0.765	0.830	0.651	4.0M

Table 6. Classification results on the held-out test set.

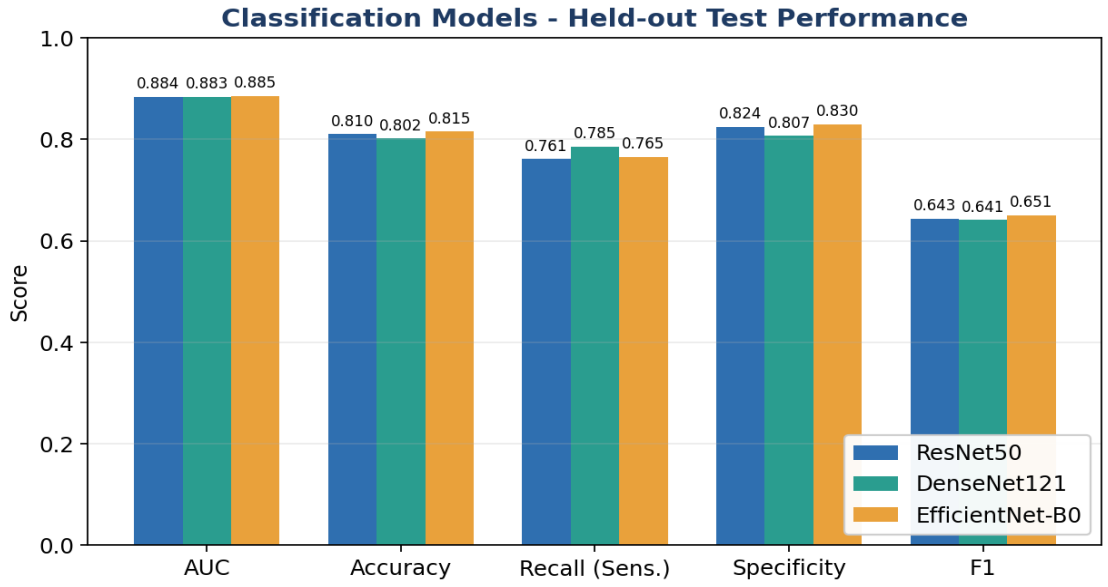

Figure 8. Classification metrics across the three classifiers.

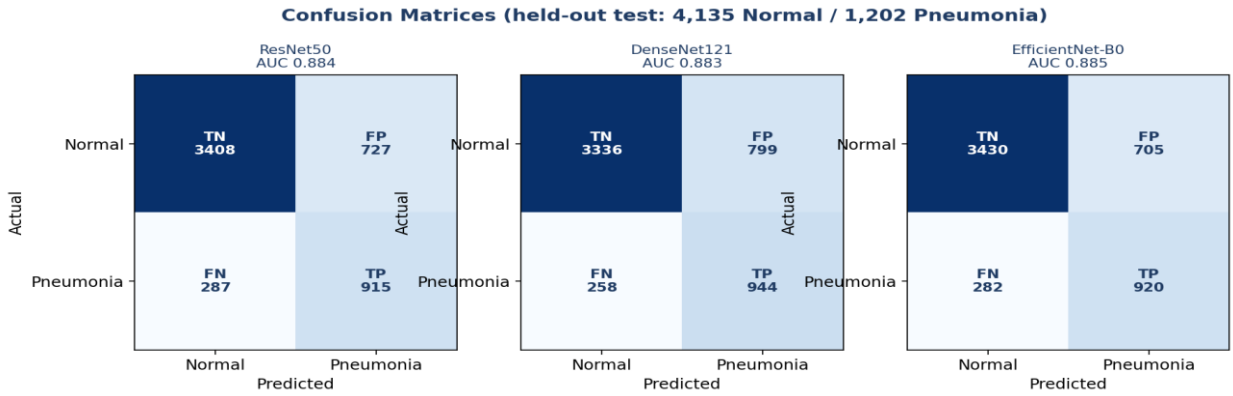

Figure 9. Confusion matrices for the three classifiers on the held-out test set.

All three classifiers perform similarly, with AUCs clustered around 0.88. EfficientNet-B0 is the strongest and also by far the smallest (4.0 million parameters), which makes it the most efficient choice for serving. The confusion matrices show the expected trade-off on an imbalanced test set: the models keep specificity high while maintaining solid sensitivity.

7.2 Detection Results

Model	mAP@0.5	mAP@[.5:.95]	Recall	Params
YOLOv8n	0.346	0.138	0.382	3.0M
Faster R-CNN	0.381	0.124	0.812	41.3M

Table 7. Detection results on the held-out test set.

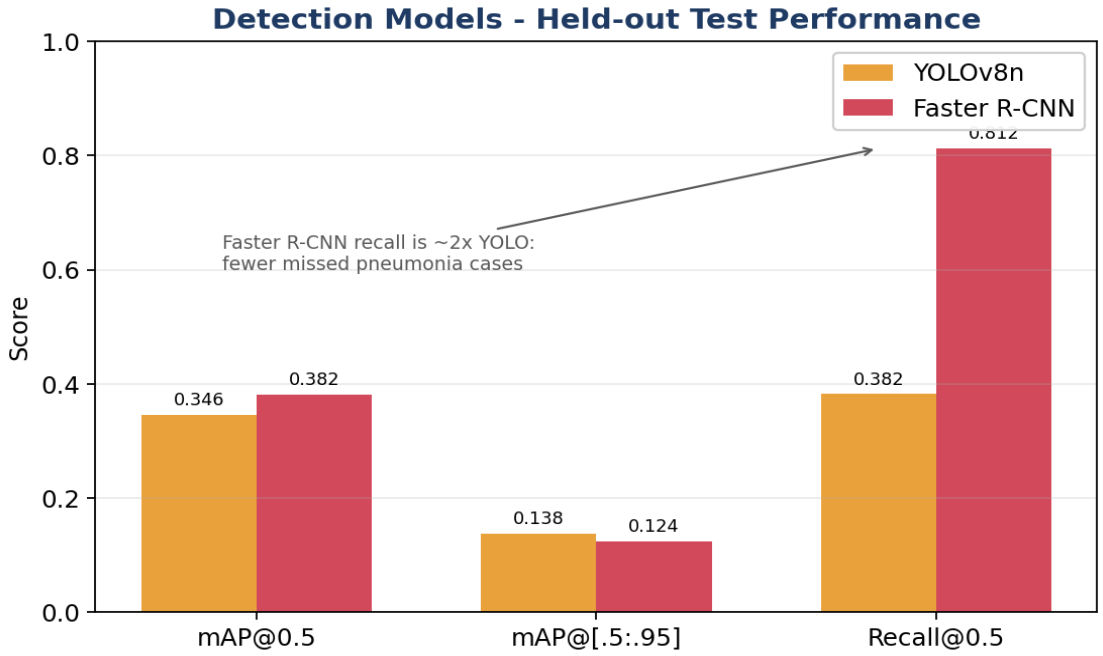

Figure 10. Detection metrics: YOLOv8n versus Faster R-CNN.

The two detectors reach a similar mAP@0.5, but they differ sharply on recall: Faster R-CNN recalls 0.812 of pneumonia regions versus 0.382 for YOLOv8n. Because a missed pneumonia is the dangerous error, we deploy Faster R-CNN as the default detector despite its larger size. This is a clear example of choosing a model on the clinically meaningful metric rather than on a headline number.

7.3 Optimization: A Negative but Useful Result

Our PSO, GWO, and SA hyperparameter search ran on a deliberately cheap proxy so it could fit a single GPU session. That proxy turned out to be too noisy to rank configurations reliably: the settings it preferred improved the proxy score but hurt the full retrain for most models. We therefore select the validation-best checkpoint per model rather than the search-suggested one. The lesson, that lightweight proxy search does not automatically beat a strong, carefully tuned baseline, is a genuine and defensible engineering finding.

7.4 Calibration Against the Literature

Our numbers are honest and competitive. A classification AUC near 0.88 is in line with established work; the well-known CheXNet reported an AUC of 0.768 for pneumonia on a different dataset. Reported RSNA detection results typically land between 0.32 and 0.39 mAP@0.5, so our Faster R-CNN at 0.381 is on par with engineered solutions. We deliberately avoided the inflated near-perfect scores that appear when the data leaks between train and test at the patient level.

7.5 Explainability Results

Because the system attaches a visual explanation to every prediction, we can confirm that the models attend to the lungs rather than to irrelevant artifacts. For a pneumonia-positive case, Figure 11 places the original radiograph next to four classifier saliency maps, all of which concentrate on the affected lung field. Figure 12 contrasts the two detectors on a case where Faster R-CNN localizes the opacity that YOLOv8 misses, with two detector-specific explanations. Figure 13 shows a normal case that EfficientNet-B0 correctly clears with high confidence.

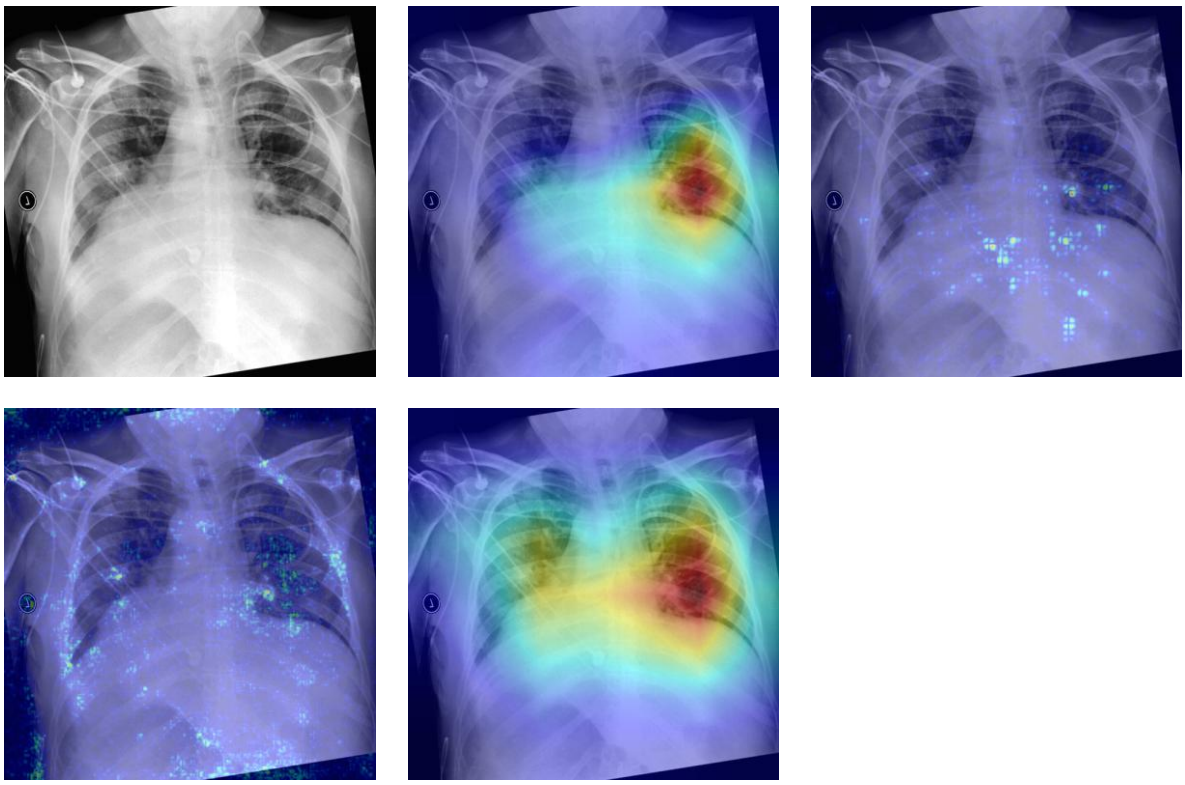

Figure 11. Classifier explainability for a pneumonia-positive case: the original X-ray followed by Grad-CAM, Integrated Gradients, GradientSHAP, and Score-CAM heatmaps, all focusing on the affected lung.

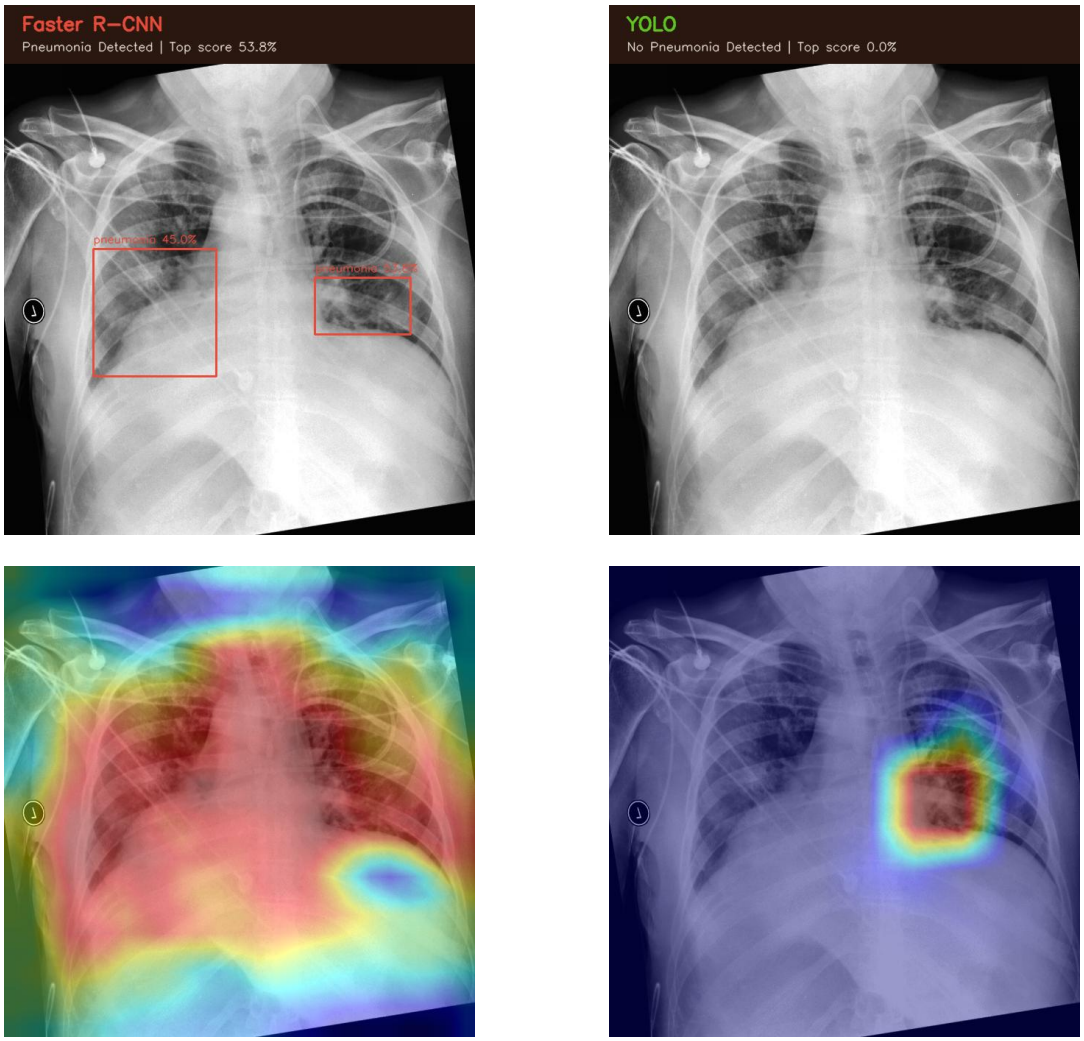

Figure 12. Detector explainability: Faster R-CNN localizes the opacity (top-left) on a case where YOLOv8 produces no box (top-right); Eigen-CAM and occlusion sensitivity (bottom) confirm the salient region.

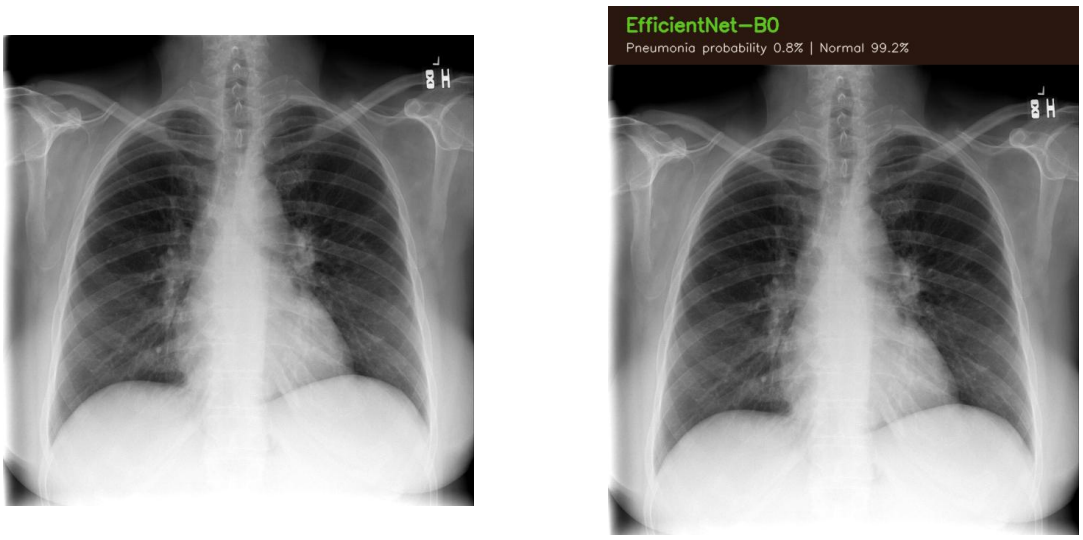

Figure 13. A normal chest X-ray (left) correctly predicted as non-pneumonia with high confidence by EfficientNet-B0 (right).

These explanations are generated live with every prediction and are best-effort: if a particular method fails on a given image it is skipped rather than blocking the reading, so the doctor always receives a result with whatever explanations succeeded.

7.6 Limitations

- Performance is bounded by the difficulty and label noise of the RSNA dataset and may not transfer to other populations or equipment without re-validation.
- The system does not yet detect out-of-distribution inputs, such as a non-chest image, beyond basic format checks.
- Load testing and production monitoring are not yet in place, so we report architecture-level rather than measured scalability.

Chapter 8 - Ethics, Compliance, and Standards

8.1 Ethical Risks Assessed and Mitigated

- Bias and fairness: the model inherits the RSNA population, so it may underperform on under-represented groups; we flag this and recommend subgroup validation before clinical use.
- Patient safety: the tool is decision-support only and never makes an autonomous diagnosis; the clinician confirms every result, which is the primary safeguard against a wrong output.
- Over-reliance: confidence scores and explainability heatmaps are shown precisely so the doctor can challenge the model rather than trust it blindly.
- Privacy: patient data is access-controlled per user, and inference runs on image pixels rather than on identity metadata.

8.2 Data Handling and Consent

Passwords are hashed with bcrypt and never stored in plaintext. Access to patient records and analyses is restricted to the owning doctor at the query level. In a real deployment the operating clinic is responsible for obtaining patient consent and for compliance with local data-protection law; the system is designed to support that with per-user isolation and an auditable, persisted record of every analysis.

8.3 Standards Followed and Why

- OWASP secure-development guidance informed authentication, input validation, and secret handling, because the system stores sensitive health data.
- GDPR-style data-minimization and access-control principles informed the database and authorization design, since the platform processes personal and health information.
- Responsible-AI practice (human-in-the-loop, explainability, honest evaluation) was applied throughout, because an opaque or overstated medical model is unsafe.

Each standard was selected for its direct relevance to a clinical, data-sensitive application and was applied concretely in the design, from hashed credentials and scoped queries to mandatory explainability on every prediction.

Chapter 9 - Conclusions and Future Work

9.1 Conclusions

This project delivered a complete, working Chest X-ray Pneumonia Detection System: five rigorously evaluated deep-learning models served through a secure, explainable, full-stack web application. We met our objectives, train and evaluate honestly, deliver a usable multi-user product, explain every prediction, and keep the work reproducible. The strongest classifier reached an AUC of 0.886 and the deployed detector reached a recall of 0.812, results that are competitive with the literature and were obtained without data leakage.

9.2 Lessons Learned

- Honest evaluation matters more than a high number; finding and fixing the data leak was one of the most valuable steps in the project.
- Choosing a model on the clinically meaningful metric (recall) can mean preferring a larger, slower architecture, and that can be the right call.
- Delivering a model as a product is a large engineering effort in its own right: security, persistence, explainability, and reproducibility all take real work.

9.3 Future Work

- Containerize the system with Docker Compose and add a CI/CD pipeline for one-command, reproducible deployment.
- Add structured logging, metrics, and monitoring, including model-drift detection in production.
- Add out-of-distribution detection so non-chest or low-quality images are rejected before inference.
- Extend from binary screening toward multi-finding detection and validate on additional, more diverse datasets.
- Pursue the clinical validation and regulatory steps required to move from decision support toward a certified tool.

References

[1] P. Rajpurkar et al., "CheXNet: Radiologist-level pneumonia detection on chest X-rays with deep learning," arXiv:1711.05225, 2017.

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137-1149, 2017.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016, pp. 779-788.

[4] G. Jocher et al., "Ultralytics YOLO," GitHub Repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] Radiological Society of North America, "RSNA Pneumonia Detection Challenge," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/competitions/rsna-pneumonia-detection-challenge

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016, pp. 770-778.

[7] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in Proc. IEEE CVPR, 2017, pp. 4700-4708.

[8] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. ICML, 2019, pp. 6105-6114.

[9] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE ICCV, 2017, pp. 618-626.

[10] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in Proc. ICML, 2017, pp. 3319-3328.

[11] J. Kennedy and R. Eberhart, "Particle swarm optimization," in Proc. IEEE ICNN, 1995, pp. 1942-1948.

[12] S. Mirjalili, S. M. Mirjalili, and A. Lewis, "Grey wolf optimizer," Advances in Engineering Software, vol. 69, pp. 46-61, 2014.

[13] S. Ramirez, "FastAPI," 2018. [Online]. Available: https://fastapi.tiangolo.com/

[14] Google, "Angular," 2024. [Online]. Available: https://angular.dev/

[15] MongoDB Inc., "MongoDB," 2024. [Online]. Available: https://www.mongodb.com/

[16] OWASP Foundation, "OWASP Top 10 Web Application Security Risks," 2021. [Online]. Available: https://owasp.org/www-project-top-ten/

Appendix 1 - User Guide

A1.1 System Requirements

- Python 3.12, Node.js 20+ and npm, and a running MongoDB instance (local or MongoDB Atlas).
- Roughly 1 GB of disk for the model weights (tracked with Git LFS).

A1.2 Installation and Setup

Backend: create a virtual environment, install requirements from Backend/requirements.txt, copy .env.example to .env and set the secret key and MongoDB URL, then start the API with "python main.py" or "uvicorn main:app". Frontend: from the Frontend folder run "npm install" and then "npm start" for development or "npm run build" for a production build. Ensure Git LFS is installed so the model weights are fetched on clone.

A1.3 Using the System

- Register a doctor account and log in.
- Create or select a patient record.
- Open the upload page and choose a chest X-ray image; a preview is shown.
- Submit for analysis and wait on the processing screen.
- Review the result: the annotated image, the pneumonia decision, the confidence score, and the heatmap.
- Find the saved analysis later in the patient history.

The screenshots below walk through the entry and history screens; the dashboard, the upload page, and the result screen are shown earlier in Figures 5 and 6.

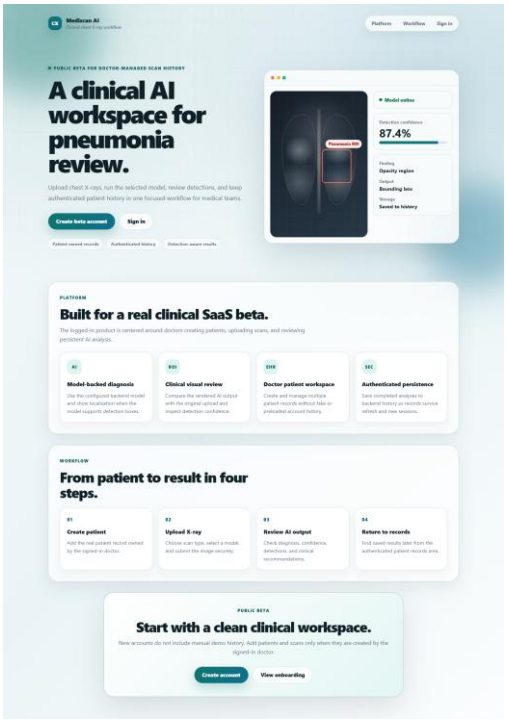

Figure 14. The public welcome screen, the entry point before signing in.

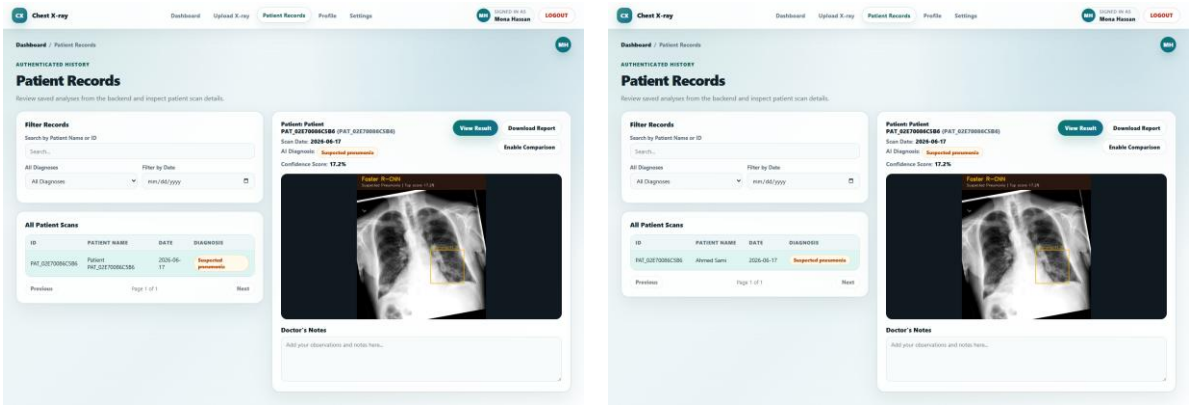

Figure 15. The patient records list (left) and the detailed view of a past analysis (right).

A1.4 Troubleshooting

- If the backend fails to start, confirm MongoDB is running and the MONGODB_URL in .env is correct.
- If a model fails to load, confirm Git LFS pulled the .pt files into Backend/model_assets.
- If the browser cannot reach the API, confirm the backend port and the CORS_ORIGINS setting.

Appendix 2 - Supporting Material

A2.1 Lessons Learned and Challenges

The most instructive challenge was a preprocessing data leak that produced near-perfect early scores; fixing it with patient-wise splitting and uniform image processing taught the team to trust the protocol over the number. Serving several large models efficiently, and making detector predictions explainable, were the main engineering challenges, solved with a single-load model registry and detector-appropriate saliency methods (Eigen-CAM and occlusion sensitivity).

A2.2 Potential Improvements

- Containerization, CI/CD, and production monitoring.
- A feedback loop that turns doctor confirmations and corrections into new training labels.
- Model quantization to lower inference latency and memory.

A2.3 Repository and Contribution Summary

The complete source code, model assets, documentation, and these deliverables are available in the project GitHub repository. The team is split into an AI track (Moamen Elsayed Elsharkawy, Habiba Ayman Amin) responsible for the data pipeline, model training, evaluation, and explainability, and a full-stack track (Ahmed Gamal Abdelfattah, Sara Mostafa Ali) responsible for the Angular frontend, the FastAPI backend, the database, and integration. Individual contributions are reflected in the Git commit history.